

Es.1 INTERPRETE

Def: L = linguaggio di prog.

Prog^L = programmi in L

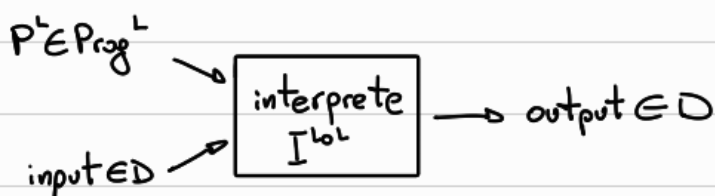
$P^L = P^L \in \text{Prog}^L$. $P^L: D \rightarrow D \quad \forall \text{input} \in D$. $P^L(\text{input}) = \text{output} \in D$

D = insieme di dati manipolati dai programmi in L

Un interprete per L (scritto in L_0) è un prog. $I^{L_0L}: (\text{Prog}^{L_0L} \times D) \rightarrow D$

$I^{L_0L}(P^L \times \text{input}) \rightarrow \text{output} = P^L(\text{input}) \rightarrow \text{output}$

macchina universale: $I^{L_0L}(P^L \times \text{input}) = P^L(\text{input})$



Un interprete è un programma scritto in un linguaggio L_0 che legge un programma P^L scritto in L e ne esegue direttamente le singole istruzioni passo dopo passo sui dati di input per restituire l'output

begin

go := true

while go do

FETCH (opcode, opinfo) at PC;

DECODE (opcode, opinfo);

if opcode needs arguments then

FETCH (args);

case opcode of

op1 EXECUTE (op1, args);

op2

:

HALT : go := false;

} operazioni:
possibili

if opcode has result then
 STORE (result);
 PC += size (opcode);

Es. 2 induzione

$$\forall n \in \mathbb{N} . n > 2 \quad n^2 > 2n+1$$

Caso base: $n=3 \quad 3^2 > 2 \cdot 3 + 1 \quad ; \quad 9 > 6 + 1 \quad ; \quad 9 > 7 \quad \checkmark$

Caso ind.: Hp. ind. $n^2 > 2n+1$
 $(n+1)^2 > 2(n+1)+1 \quad ; \quad (n+1)^2 > 2n+3$

1) $n^2 > 2n+1 \quad ; \quad n^2 + \underline{(2n+1)} > 2n+1 + \underline{(2n+1)} \quad ; \quad (n+1)^2 > 4n+2$

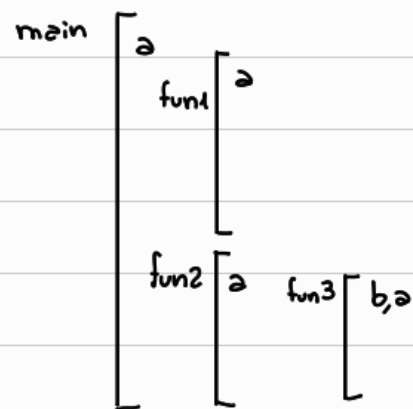
2) $4n+2 > 2n+3 \quad \rightarrow \quad 4n+2 = (2n+3) + (2n-1) \quad ; \quad 2n+3 + 2n-1 > 2n+3$

$$(n+1)^2 > 4n+2 > 2n+3 \quad \checkmark$$

Es. 3 scoping statico e dinamico

```
{ int a;
  void fun1() { a = a + 2; }
  void fun2() {
    int a = 5;
    void fun3(int b) {
      int a = b + 4;
      fun1(); }
    fun1();
    fun3(4); }
```

```
a = 4;
fun2(); }
```



profondità dichiarazioni:

$$Sd(main) = 0$$

$$Sd(fun1) = 1$$

$$Sd(fun2) = 1$$

$$Sd(fun3) = 2$$

seq. chiamate:

main $\rightarrow$ fun2 $\rightarrow$ fun1
 $\searrow$ fun3 $\rightarrow$ fun1

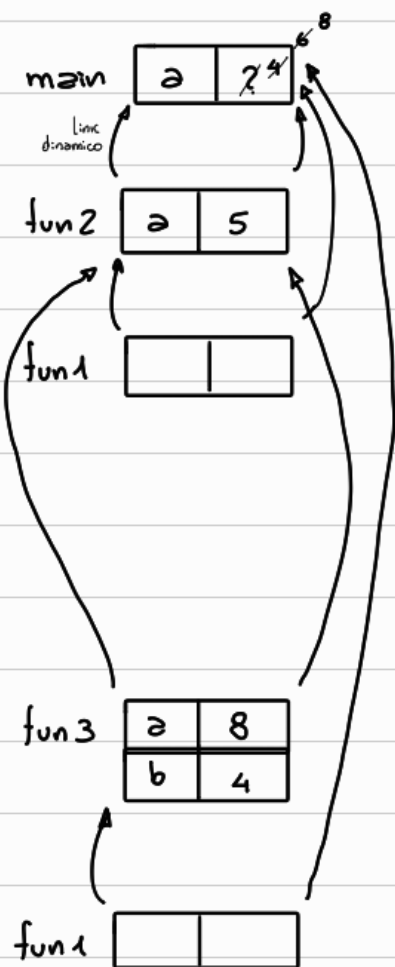
link statico $K = Sd(chiamante) - Sd(chiamato) + 1$

di volte da risalire lungo la catena statica
per trovare il genitore statico

risoluzione riferimenti $N_a = Sd(uso) - Sd(dichiarante)$

di volte da risalire lungo la catena statica
per trovare RDA del blocco che definisce a

Scoping statico



esecuzione main: $a = 4;$

link statico $K_a = Sd(main) - Sd(fun2) + 1$
 $= 0 - 1 + 1 = 0 \rightarrow CS(fun2) = main$

link statico $K_a = Sd(fun2) - Sd(fun1) + 1$
 $= 1 - 1 + 1 = 1 \rightarrow CS(fun1) = main$

esecuzione di fun1: $a = a + 2;$

$N_a = Sd(fun1) - Sd(main)$
 $= 1 - 0 = 1 \rightarrow a = 4$

link statico $K_a = Sd(fun2) - Sd(fun3) + 1$
 $= 1 - 2 + 1 = 0 \rightarrow CS(fun3) = fun2$

esecuzione di fun3: $a = b + 4$

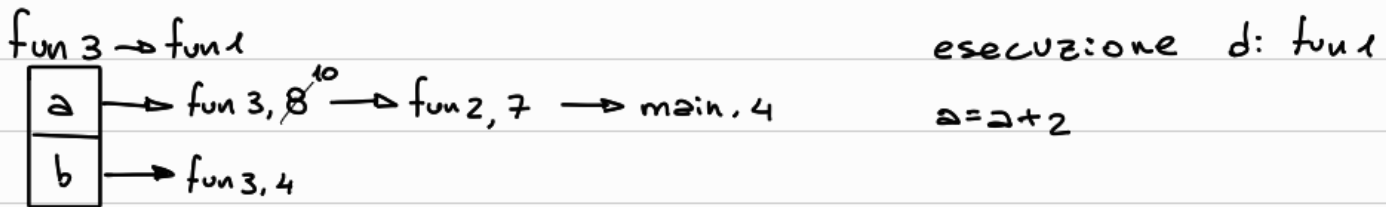
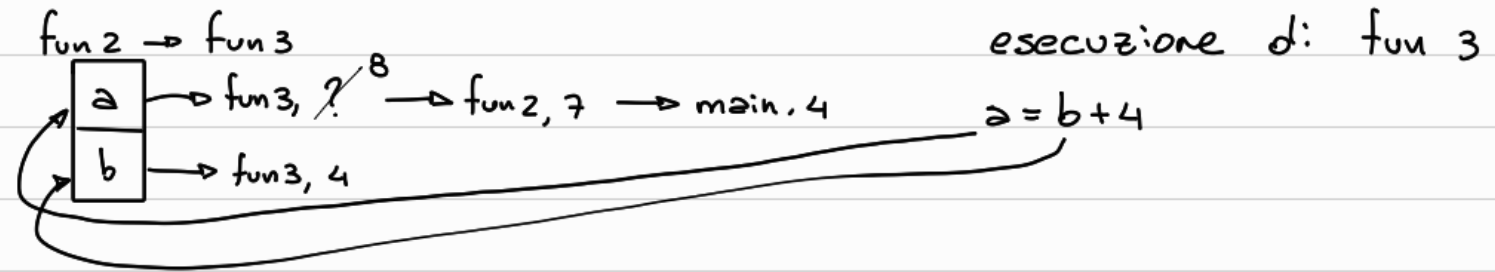
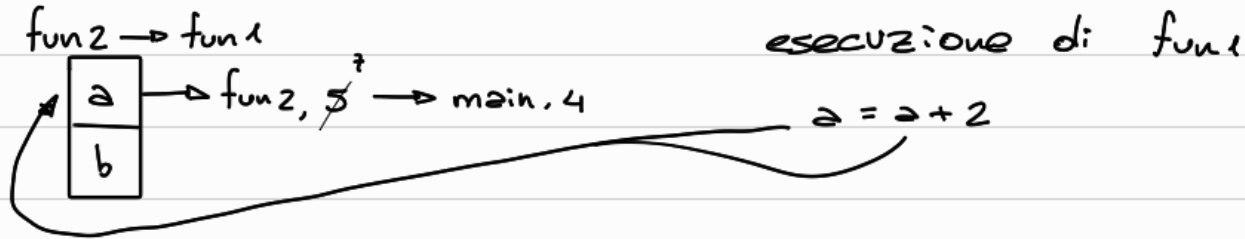
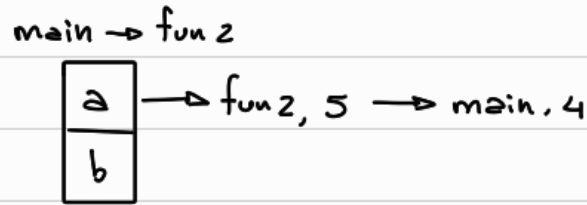
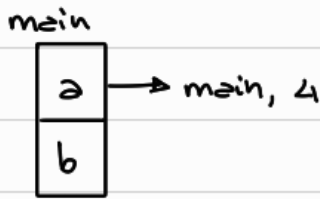
link statico $K_a = Sd(fun3) - Sd(fun1) + 1$
 $= 2 - 1 + 1 = 2$

esecuzione di fun1: $a = a + 2;$

$N_a = Sd(fun1) - Sd(main) = 1 - 0 = 1 \rightarrow a = 6$

Dopo la chiamata tutte le chiamate terminano e lo stack deallocato

Scoping dinamico



Es. 4 scoping, binding

int x=3; int y=2;

int fun1 (int in) { return x+y+in; }

int fun2 (int h(int b)) {

int y=4; int x=4;

return h(5)+x; }

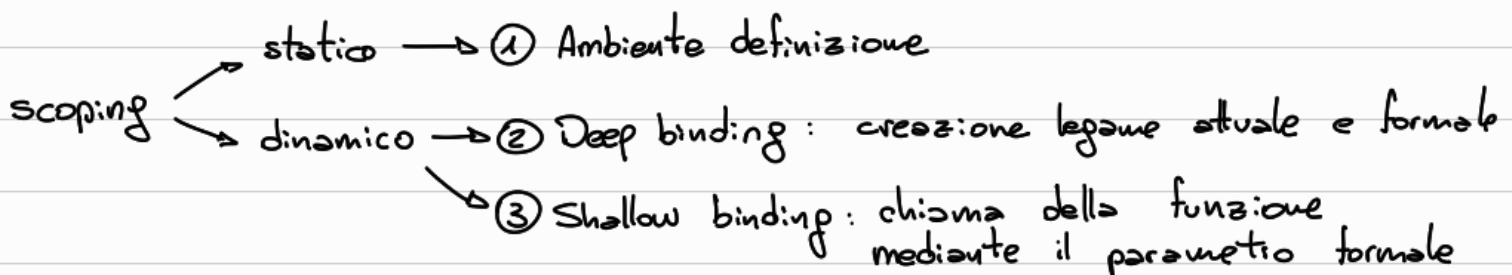
{ int x=6; int y=3;

int z = fun2(fun1); }

← ① Ambiente valido al momento della definizione

← ③ Ambiente valido quando la procedura viene effettivamente eseguita

← ② Ambiente valido al momento del passaggio



① Scoping statico

chiamata di funz

$z = \text{funz}(\text{funz})$

$z = 14$

chiamata di h

$h(5) + x$

$10, \text{locale}$

$10 + 4 = 14$

esecuzione di h

return $x + y + \text{in}$

$x = 3, y = 2, \text{locale}$

$3 + 2 + 5 = 10$

② Scoping dinamico + Deep binding

chiamata di funz

$z = \text{funz}(\text{funz})$

$z = 18$

chiamata di h

$h(5) + x$

$14, \text{locale}$

$14 + 4 = 18$

esecuzione di h

return $x + y + \text{in}$

$x = 6, y = 3, \text{locale}$

$6 + 3 + 5 = 14$

③ Scoping dinamico + Shallow binding

chiamata di funz

$z = \text{funz}(\text{funz})$

$z = 17$

chiamata di h

$h(5) + x$

$13, \text{locale}$

$13 + 4 = 17$

esecuzione di h

return $x + y + \text{in}$

$x = 4, y = 4, \text{locale}$

$4 + 4 + 5 = 13$

Es. 5 ricorsione e ricorsione in coda

```
lista function (lista list) {  
  if (length(list) = 0) then return ();  
  else return concat (2 + car(list), function(cdr(list)));  
}
```

Definizione di ricorsione:

Metodo alternativo alla costruzione di procedure iterative.

È definita procedura ricorsiva se è definita in funzione di se stessa, ovvero all'interno della procedura esiste una chiamata alla procedura stessa.

Es. 6 derivazioni:

$\rho = [x \mapsto \text{true}, y \mapsto l_y]$ l_y variabile booleana

$\sigma = [l_y \mapsto \text{false}]$

$y := x \text{ and } y$ $\sigma(\rho \vdash \langle x := x \text{ and } y, \sigma \rangle \rightarrow \dots)$

$\Delta = [x \mapsto \text{bool}, y \mapsto \text{boolloc}]$

$$\frac{\Delta \vdash x : \text{bool} \quad \Delta \vdash y : \text{bool}}{\Delta \vdash x \text{ and } y : \text{bool}} \quad \Delta(y) = \text{boolloc} \rightarrow \begin{cases} \text{se } x \text{ and } y \text{ e'} \\ \text{bool allora } y := x \text{ and } y \\ \text{e' ben formato} \end{cases}$$

esecuzione

$$\frac{\rho \vdash \langle x \text{ and } y, \sigma \rangle \xrightarrow{*} \langle \kappa, \sigma \rangle}{\rho \vdash \langle y := x \text{ and } y, \sigma \rangle \rightarrow \langle y := \kappa, \sigma \rangle}$$

valutazione di x : $\frac{\rho \vdash \langle x, \sigma \rangle \rightarrow \langle \text{true}, \sigma \rangle}{\rho \vdash \langle x \text{ and } y, \sigma \rangle \rightarrow \langle \text{true and } y, \sigma \rangle} \quad \rho(x) = \text{true}$

val. di y : $\frac{\rho \vdash \langle y, \sigma \rangle \rightarrow \langle \text{false}, \sigma \rangle}{\rho \vdash \langle \text{true and } y, \sigma \rangle \rightarrow \langle \text{true and false}, \sigma \rangle} \quad \begin{aligned} \rho(y) &= l_y \\ \sigma(l_y) &= \text{false} \end{aligned}$

Soluzione regola

$$\frac{\rho \vdash \langle \text{true and false}, \sigma \rangle \rightarrow \langle \text{false}, \sigma \rangle}{\rho \vdash \langle y := \text{true and false} \rangle \rightarrow \langle y := \text{false}, \sigma \rangle}$$

$\rho \vdash \langle y := \text{false}, \sigma \rangle \rightarrow \rho(y) = l_y \rightarrow \sigma[l_y \mapsto \text{false}]$

Il valore nella locazione l_y della variabile y viene "modificato" in false.